Comprehensive Formalization and Algorithmic Taxonomy of Contiguous Subarray Problems: Prefix State Hashing, Sliding Window Invariants, AtMost Duality, Kadane Dynamic Programming, Monotonic Stacks, Bitwise Monotonicity, and 2D Submatrix Reductions

Abstract

Contiguous subarray problems form a fundamental computational primitive across sequence optimization, dynamic programming, and technical coding platforms such as LeetCode. A central challenge in sequence processing is evaluating metrics—such as sum, product, parity, distinct element counts, minimums, maximums, or bitwise combinations—over bounded contiguous segments. Naive brute-force enumeration evaluates quadratic $\mathcal{O}(N^2)$ candidate windows, incurring $\mathcal{O}(N^3)$ time. This research paper presents an exhaustive, publication-grade taxonomy and algorithmic analysis of contiguous subarray problems. We establish nine foundational paradigms that systematically govern all optimal solutions: (1) Prefix-Sum Space and Hash-Map Equivalence Classes ($\mathcal{O}(N)$ exact sum, remainder, and zero-sum balance matching), (2) Two-Pointer Sliding Window Contractions ($\mathcal{O}(N)$ greedy optimization on monotonic non-negative domains), (3) The AtMost(K) Set-Theoretic Duality (Exact-K count decomposition via inclusion-exclusion), (4) Dynamic Programming & Kadane's Local State Optimization ($\mathcal{O}(N)$ maximum, minimum, and product subarray state transitions and circular topologies), (5) Monotonic Stack & Element Contribution Mechanics ($\mathcal{O}(N)$ range boundaries, sum of subarray minimums, and min-products), (6) Sliding Window with Monotonic Extremum Queues ($\mathcal{O}(1)$ amortized window min/max bounds), (7) Non-Monotonic Prefix Spaces with Inequalities ($\mathcal{O}(N \log N)$ merge sort divide-and-conquer and $\mathcal{O}(N)$ monotonic deques on prefix sums), (8) Bitwise Monotonicity & Bounded State Transitions ($\mathcal{O}(30N)$ accumulation limits for OR and AND), and (9) Two-Dimensional Submatrix Extensions ($\mathcal{O}(R^2C)$ row-pair projection and 1D reduction). For each paradigm, we present mathematical invariant proofs, complexity theorems, edge-case taxons, granular execution trace tables, vector TikZ architectural diagrams, complete compilable Java reference implementations, and synthesize a 10-second diagnostic decision flowchart and master LeetCode taxonomy matrix.

Index Terms

Contiguous Subarrays, LeetCode Master Patterns, Prefix Sum Hashing, Sliding Window, AtMost Duality, Kadane Algorithm, Monotonic Stack, Element Contribution Principle, Monotonic Deque, Bitwise Monotonicity, Divide and Conquer, 2D Submatrix Reduction

1 Introduction, Mathematical Foundations, and Combinatorics

Contiguous subarray problems represent one of the most intellectually rich, practically critical, and frequently tested algorithmic domains across theoretical computer science, quantitative finance, and elite technical interviews at Tier-1 technology companies (Google, Meta, Amazon, Apple, Netflix). At its core, sequence processing requires evaluating quantitative metrics—such as contiguous summation, cumulative product, parity distributions, discrete frequencies, structural bounds, minimums, maximums, and bitwise combinations—over contiguous index ranges of an input array.

While naive brute-force examination of all possible contiguous windows requires evaluating quadratic $\mathcal{O}(N^2)$ candidate subarrays and incurring catastrophic $\mathcal{O}(N^3)$ computational time, optimal algorithms achieve linear $\mathcal{O}(N)$ or linearithmic $\mathcal{O}(N \log N)$ execution time by exploiting structural invariants, algebraic equivalences, monotonic state transitions, and memory-efficient data structures.

This research paper presents an encyclopedic, publication-grade algorithmic compendium and formal taxonomy of contiguous subarray problems. We establish nine foundational computational paradigms that completely govern the design of optimal subarray algorithms:

1) Prefix-Sum Space and Hash-Map Equivalence Classes (§2): Decomposing contiguous subarray metrics into prefix state differences, enabling exact sum matching, divisibility classes, zero-sum optimizations, and domain transformations in $\mathcal{O}(N)$ time.

2) Sliding Window and Two-Pointer Search Space Pruning (§3): Exploiting monotonic contraction properties across non-negative domains to prune candidate states from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, including fixed-window and variable-window metrics.
3) The AtMost(K) Set-Theoretic Duality (§4): Solving exact-K count constraints over non-monotonic cardinality boundaries via set-theoretic decomposition: $\text{Exact}(K) = \text{AtMost}(K) - \text{AtMost}(K-1)$.
4) Dynamic Programming and Kadane's Local State Optimization (§5): Formalizing optimal sub-structure for maximum/minimum subarray sums, product propagation under multiplicative sign inversion, circular topology transformations, and single-element deletion models.
5) Monotonic Stack and Element Contribution Mechanics (§6): Inverting search focus from subarray iteration to element contribution, deriving exact left/right boundary ranges (PSE and NSE) to evaluate sum of subarray minimums, maximums, and min-products in $\mathcal{O}(N)$ time.
6) Sliding Window with Monotonic Extremum Queues (§7): Tracking running window minimums and maximums in $\mathcal{O}(1)$ amortized time via monotonic double-ended queues.
7) Divide-and-Conquer, Range Trees, and Monotonic Deques on Negative Prefixes (§8): Resolving non-monotonic prefix spaces for range sum bounds ($\mathcal{O}(N \log N)$ merge-sort divide and conquer) and shortest target sums with negative numbers ($\mathcal{O}(N)$ monotonic deque on prefix sums).
8) Bitwise Monotonicity and Bounded State Transitions (§9): Exploiting the 32-bit accumulation boundary of bitwise OR and AND operations to constrain distinct subarray bitwise states to $\mathcal{O}(30N)$.
9) Two-Dimensional Submatrix Extensions (§10): Generalizing 1D contiguous subarray principles to 2D matrices via row-pair projections and column reductions.

1.1 Formal Definitions and Notation

Let $A = [a_0, a_1, \ldots, a_{N-1}]$ be a sequence of N real or integer elements indexed from 0 to $N-1$.

Definition 1 (Contiguous Subarray). A contiguous subarray $A[i \ldots j]$ defined by start boundary i and end boundary j (where $0 \le i \le j < N$) is the ordered subsequence of elements:

$$A[i \ldots j] = (a_i, a_{i+1}, \ldots, a_j) \tag{1}$$

A subarray preserves strict contiguous spatial locality: no intermediate elements within $[i, j]$ may be omitted.

Theorem 2 (Cardinality of Subarray Space). The total number of non-empty contiguous subarrays in an array of size N is exactly $\frac{N(N+1)}{2} = \mathcal{O}(N^2)$.

Proof. We establish this fundamental result through two distinct combinatorial proofs:

1) Method 1: Partition by Subarray Length: Let L denote the length of a subarray, where $1 \le L \le N$. For a fixed length L, the valid starting index i satisfies $0 \le i \le N - L$. Thus, there are exactly $N - L + 1$ distinct contiguous subarrays of length L. Summing over all possible lengths $L \in [1, N]$:

$$\sum_{L=1}^{N} (N - L + 1) = N + (N-1) + (N-2) + \cdots + 1 = \frac{N(N+1)}{2} \tag{2}$$

2) Method 2: Combinatorial Boundary Selection: An array of length N contains $N+1$ discrete boundary partition points: $0, 1, 2, \ldots, N$. Any non-empty contiguous subarray is uniquely defined by choosing an unordered pair of distinct partition points (u, v) such that $0 \le u < v \le N$, where u represents the left boundary index and v represents the right boundary index. The number of ways to choose 2 distinct points from $N+1$ points is:

$$\binom{N+1}{2} = \frac{(N+1)!}{2!(N-1)!} = \frac{N(N+1)}{2} \tag{3}$$

Both methods confirm that the candidate search space has exact cardinality $\frac{N(N+1)}{2} = \mathcal{O}(N^2)$. □

Theorem 3 (The Element Contribution Principle). In the sum of all $\frac{N(N+1)}{2}$ contiguous subarrays, element a_i (at index i, where $0 \le i < N$) appears in exactly $(i+1)(N-i)$ subarrays. Consequently:

$$\sum_{0 \le i \le j < N} \sum_{k=i}^{j} a_k = \sum_{i=0}^{N-1} (i+1)(N-i) \cdot a_i \tag{4}$$

Proof. A contiguous subarray $A[L \ldots R]$ contains the element a_i if and only if the left boundary L satisfies $0 \le L \le i$ and the right boundary R satisfies $i \le R < N$.

- The number of valid left boundary choices is $i - 0 + 1 = i + 1$.
- The number of valid right boundary choices is $(N-1) - i + 1 = N - i$.

TABLE 1
Formal Comparison of Subarray, Subsequence, and Subset Spaces

Structure Type	Contiguity Invariant	Relative Order	Cardinality	Production Example
Subarray	Strictly contiguous (no gaps)	Strictly preserved	$\frac{N(N+1)}{2} = \mathcal{O}(N^2)$	$A = [1,2,3] \implies$ [1], [2], [3], [1, 2], [2, 3], [1, 2, 3]
Subsequence	Gaps permitted between elements	Strictly preserved	$2^N - 1 = \mathcal{O}(2^N)$	$A = [1,2,3] \implies [1,3]$ is valid subsequence, but not subarray
Subset	Arbitrary selection	Irrelevant	$2^N = \mathcal{O}(2^N)$	$A = [1,2,3] \implies \{3,1\}$ is valid subset

By the fundamental counting principle of combinatorics, the choices of left and right boundaries are completely independent. Therefore, the total number of subarrays containing a_i is precisely the product of choices:

$$\text{Frequency}(a_i) = (i+1)(N-i) \tag{5}$$

Multiplying each element a_i by its frequency and summing over all $i \in [0, N-1]$ yields the grand sum across all $\mathcal{O}(N^2)$ subarrays in optimal $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space. □

Definition 4 (Prefix Sum Sequence). The prefix sum sequence P of an array A is defined for $r \in [0, N-1]$ as:

$$P[r] = \sum_{k=0}^{r} a_k \quad \text{with base definition } P[-1] = 0 \tag{6}$$

Lemma 5 (Subarray Sum Decomposition). The sum of elements in contiguous subarray $A[i \dots j]$, denoted $S(i,j)$, equals the difference of prefix sums:

$$S(i,j) = \sum_{k=i}^{j} a_k = P[j] - P[i-1] \quad \text{for } 0 \le i \le j < N \tag{7}$$

2 Paradigm 1: Prefix-Sum Space and Hash-Map Equivalence Classes

The algebraic identity $S(i,j) = P[j] - P[i-1]$ allows us to recast the search for contiguous subarrays into an equivalent two-sum lookup on prefix sums: finding i such that $P[i-1] = P[j] - K$.

2.1 Why Sliding Window Fails on Negative Numbers

A classic interview pitfall is attempting to solve subarray sum problems involving negative integers using two-pointer sliding windows. A two-pointer sliding window strictly requires monotonicity: advancing the right pointer must monotonically increase (or decrease) the window metric, and advancing the left pointer must monotonically invert that change. When negative numbers are present, adding an element can either increase or decrease the sum, destroying monotonicity and invalidating greedy window contractions. Prefix sum hashing is the exact mathematical remedy.

2.2 Target Sum Matching ($P[j] - P[i-1] = K$)

2.2.1 Subarray Sum Equals K (LeetCode 560)

We iterate j from 0 to $N-1$, maintaining cumulative running prefix $P[j]$ and a frequency hash map M. For each j, the number of subarrays ending at j with sum K equals the frequency of prefix value $(P[j] - K)$ observed prior to index j.

- Mandatory Base Case Guard: $M[0] = 1$. This accounts for subarrays starting at index 0 ($A[0 \dots j]$), where $P[j] = K \implies P[j] - K = 0 = P[-1]$. Omitting this base case silently drops all valid prefix subarrays starting at index 0.

2.2.2 Maximum Size Subarray Sum Equals K (LeetCode 325)

When seeking the maximum length of a subarray with sum K, the hash map stores the earliest occurrence index of each prefix sum: $M[P[i-1]] = i-1$. Upon encountering $P[j]$, if $(P[j] - K) \in M$, candidate length is $j - M[P[j] - K]$. To maximize length, existing keys are never overwritten.

2.3 Modulo and Remainder Equivalence Classes ($S(i,j) \pmod K = 0$)

2.3.1 Subarray Sums Divisible by K (LeetCode 974)

By number theory, $S(i,j) \equiv 0 \pmod K \iff P[j] \equiv P[i-1] \pmod K$. Thus, any two prefix sums sharing the same remainder modulo K delineate a contiguous subarray whose sum is an exact multiple of K.

- The Negative Modulo Trap in Java: In Java, the modulo operator % retains the sign of the dividend (e.g., -7 % 5 = -2). In modular arithmetic, remainders must reside in $[0, K-1]$. The normalized positive remainder is:

$$rem = ((pref \pmod K) + K) \pmod K \tag{8}$$

TABLE 2
Detailed Step-by-Step Execution Trace for Subarray Sum Equals K (LeetCode 560)

Index j	Element a_j	Prefix Sum $P[j]$	Target Needed $P[j]-K$	Freq in Map	Total Count	Hash Map State
Base	—	0	—	—	0	$\{0:1\}$
0	3	3	$3-7=-4$	0	0	$\{0:1, 3:1\}$
1	4	7	$7-7=0$	$1 \implies A[0\ldots1]$	1	$\{0:1, 3:1, 7:1\}$
2	7	14	$14-7=7$	$1 \implies A[2\ldots2]$	2	$\{0:1, 3:1, 7:1, 14:1\}$
3	2	16	$16-7=9$	0	2	$\{\ldots, 16:1\}$
4	-3	13	$13-7=6$	0	2	$\{\ldots, 13:1\}$
5	1	14	$14-7=7$	$1 \implies A[3\ldots5]$	3	$\{\ldots, 7:1, 14:2\}$
6	4	18	$18-7=11$	0	3	$\{\ldots, 18:1\}$
7	2	20	$20-7=13$	$1 \implies A[5\ldots7]$	4	$\{\ldots, 20:1\}$

2.3.2 Continuous Subarray Sum of Length at Least 2 (LeetCode 523)

Requires $S(i,j) = m \cdot K$ and length $(j-i+1) \geq 2$. We store the earliest index of each remainder in a hash map initialized with $M[0] = -1$. When rem is seen again at index j, if $j - M[rem] \geq 2$, return true.

2.4 Domain Transformation: Equal 0s and 1s & Parity Matching

2.4.1 Contiguous Array with Equal 0s and 1s (LeetCode 525)

Transform the binary array into balance space by mapping $0 \to -1$ and $1 \to +1$. A subarray has equal 0s and 1s if and only if its transformed sum equals 0. The problem reduces directly to finding the longest subarray with sum 0 ($P[j] = P[i-1]$).

2.4.2 Number of Subarrays with Odd Sum (LeetCode 1524)

A sum is odd if and only if the parity of $P[j]$ differs from $P[i-1]$. We maintain running counts of even and odd prefix sums seen so far. At each step, if $P[j]$ is odd, it forms odd subarrays with all previous even prefixes; if $P[j]$ is even, it forms odd subarrays with all previous odd prefixes.

2.4.3 Count Subarrays With Median K (LeetCode 2488)

Transform elements: map elements $> K$ to $+1$, elements $< K$ to -1, and element $= K$ to 0. A subarray centered at K has median K if its balance sum is either 0 (odd length) or $+1$ (even length).

2.5 Compilable Java Reference Implementation: Paradigm 1

```java
import java.util.HashMap;
import java.util.Map;

public class PrefixHashSubarraySuite {

    // 1. Subarray Sum Equals K (LeetCode 560) - O(N) Time, O(N) Space
    public static int subarraySumEqualsK(int[] nums, int k) {
        Map<Long, Integer> freq = new HashMap<>();
        freq.put(0L, 1); // Mandatory Base Case: P[-1] = 0
        long pref = 0;
        int count = 0;

        for (int x : nums) {
            pref += x;
            if (freq.containsKey(pref - k)) {
                count += freq.get(pref - k);
            }
            freq.put(pref, freq.getOrDefault(pref, 0) + 1);
        }
        return count;
    }

    // 2. Subarray Sums Divisible by K (LeetCode 974) - O(N) Time, O(K) Space
    public static int subarraysDivByK(int[] nums, int k) {
        int[] remFreq = new int[k];
        remFreq[0] = 1; // Base case: remainder 0 seen once initially
        long pref = 0;
        int count = 0;

        for (int x : nums) {
            pref += x;
            int rem = (int) ((pref % k + k) % k); // Normalized positive remainder
            count += remFreq[rem];
            remFreq[rem]++;
```

```
        }
        return count;
    }

    // 3. Contiguous Array - Equal 0s and 1s (LeetCode 525) - O(N) Time, O(N) Space
    public static int findMaxLengthEqual01(int[] nums) {
        Map<Integer, Integer> firstSeen = new HashMap<>();
        firstSeen.put(0, -1); // Base case: balance 0 at virtual index -1
        int balance = 0, maxLen = 0;

        for (int j = 0; j < nums.length; j++) {
            balance += (nums[j] == 1) ? 1 : -1;
            if (firstSeen.containsKey(balance)) {
                maxLen = Math.max(maxLen, j - firstSeen.get(balance));
            } else {
                firstSeen.put(balance, j); // Retain earliest occurrence index
            }
        }
        return maxLen;
    }
}
```

3 Paradigm 2: Sliding Window and Two-Pointer Search Space Pruning

When sequence elements are strictly non-negative ($a_k \geq 0$), the prefix sum sequence $P[j]$ is monotonically non-decreasing ($P[j] \geq P[j-1]$). Expanding the right boundary R increases window sum and distinct element counts, while advancing the left boundary L decreases window metrics. This monotonicity enables two-pointer search space pruning from $\mathcal{O}(N^2)$ to amortized $\mathcal{O}(N)$.

3.1 Fixed-Size Window Pattern

When subarray length is constrained to an exact constant K, the window $[i, i+K-1]$ maintains state by sliding: adding incoming element a_{i+K-1} and dropping outgoing element a_{i-1} in $\mathcal{O}(1)$ time.

- Maximum Average Subarray I (LeetCode 643): Running sum of length K.
- Maximum Sum of Distinct Subarrays with Length K (LeetCode 2461): Frequency map or hash set alongside running sum, verifying $|set| == K$.
- Find All Anagrams in a String (LeetCode 438): Fixed window matching character frequency arrays of length equal to pattern P.

3.2 Variable Window Contraction Mechanics

3.2.1 Minimum Size Subarray Sum (LeetCode 209)

We maintain window $[L, R]$. For each R, we add $A[R]$ to $window_sum$. While $window_sum \geq Target$, we update $min_len = \min(min_len, R - L + 1)$ and contract L by subtracting $A[L]$.

Theorem 6 (Amortized Linear Complexity of Sliding Window). The two-pointer sliding window algorithm executes in $\mathcal{O}(N)$ total time complexity and $\mathcal{O}(1)$ auxiliary space.

Proof. The right pointer R increments from 0 to $N-1$ exactly N times. The left pointer L increments monotonically from 0 to $N-1$ and never decrements. Across the entire execution, L is incremented at most N times. Total pointer movements are bounded by $2N = \mathcal{O}(N)$. □

TABLE 3
Step-by-Step Execution Trace for Minimum Size Subarray Sum (LeetCode 209)

Step	Right R	Incoming a_R	Window $[L \dots R]$	Window Sum	Condition ≥ 7	Min Length Recorded
1	0	2	[2]	2	False	∞
2	1	3	[2, 3]	5	False	∞
3	2	1	[2, 3, 1]	6	False	∞
4	3	2	[2, 3, 1, 2]	8	True $\implies$ Shrink $L = 0$	$\min(\infty, 4) = 4$
4*b*	3	—	[3, 1, 2]	6	False $\implies$ $L = 1$	4
5	4	4	[3, 1, 2, 4]	10	True $\implies$ Shrink $L = 1$	$\min(4, 4) = 4$
5*b*	4	—	[1, 2, 4]	7	True $\implies$ Shrink $L = 2$	$\min(4, 3) = 3$
5*c*	4	—	[2, 4]	6	False $\implies$ $L = 3$	3
6	5	3	[2, 4, 3]	9	True $\implies$ Shrink $L = 3$	$\min(3, 3) = 3$
6*b*	5	—	[4, 3]	7	True $\implies$ Shrink $L = 4$	$\min(\mathbf{3}, \mathbf{2}) = \mathbf{2}$
6*c*	5	—	[3]	3	False $\implies$ $L = 5$	2

3.2.2 Subarray Product Less Than K (LeetCode 713)

For positive integer arrays, expanding R multiplies *prod* by $A[R]$. While $prod \geq K$, we divide *prod* by $A[L]$ and advance L.

Lemma 7 (Ending-At Count Invariant). For a valid window $[L, R]$ where all sub-windows ending at R satisfy the condition, the number of valid contiguous subarrays ending strictly at R is exactly $R - L + 1$.

Proof. The valid subarrays ending at R are $A[R \dots R], A[R-1 \dots R], \dots, A[L \dots R]$. The starting indices range from L to R, giving $(R - L + 1)$ distinct choices. □

3.3 Compilable Java Reference Implementation: Paradigm 2

```java
public class SlidingWindowSubarraySuite {

    // 1. Minimum Size Subarray Sum >= S (LeetCode 209) - O(N) Time, O(1) Space
    public static int minSubArrayLen(int target, int[] nums) {
        int L = 0, sum = 0, minLen = Integer.MAX_VALUE;
        for (int R = 0; R < nums.length; R++) {
            sum += nums[R];
            while (sum >= target) {
                minLen = Math.min(minLen, R - L + 1);
                sum -= nums[L++];
            }
        }
        return minLen == Integer.MAX_VALUE ? 0 : minLen;
    }

    // 2. Subarray Product Less Than K (LeetCode 713) - O(N) Time, O(1) Space
    public static int numSubarrayProductLessThanK(int[] nums, int k) {
        if (k <= 1) return 0;
        int L = 0, prod = 1, count = 0;
        for (int R = 0; R < nums.length; R++) {
            prod *= nums[R];
            while (prod >= k) {
                prod /= nums[L++];
            }
            count += (R - L + 1); // Ending-At Count Invariant
        }
        return count;
    }

    // 3. Max Consecutive Ones III (LeetCode 1004) - O(N) Time, O(1) Space
    public static int longestOnes(int[] nums, int k) {
        int L = 0, zeroCount = 0, maxLen = 0;
        for (int R = 0; R < nums.length; R++) {
            if (nums[R] == 0) zeroCount++;
            while (zeroCount > k) {
                if (nums[L] == 0) zeroCount--;
                L++;
            }
            maxLen = Math.max(maxLen, R - L + 1);
        }
        return maxLen;
    }
}
```

4 Paradigm 3: The AtMost(K) Set-Theoretic Duality

A frequent obstacle in sequence optimization is counting contiguous subarrays satisfying an exact condition:

- Subarrays with exactly K distinct integers (LeetCode 992).
- Binary subarrays with sum exactly K (LeetCode 930).
- Nice subarrays with exactly K odd integers (LeetCode 1248).

4.1 Why Direct Sliding Window Fails for Exact K

In sliding window algorithms, we require monotonic contraction: if a window satisfies a property, shrinking it preserves or drives it toward a boundary. However, the condition $|Distinct(A[L \dots R])| == K$ is non-monotonic. If a window has K distinct elements, shrinking L might either keep it at K (if $A[L]$ occurs elsewhere in the window) or drop it to $K - 1$. The left pointer cannot greedily advance without potentially skipping valid sub-windows.

4.2 The Master Set Duality Formula

By basic set theory, the set of subarrays satisfying an exact metric K equals the set difference between those satisfying $\leq K$ and those satisfying $\leq (K-1)$:

$$\text{Exact}(K) = \text{AtMost}(K) - \text{AtMost}(K-1) \tag{9}$$

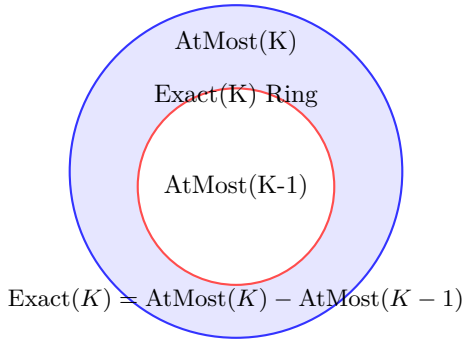


Fig. 1. The Set-Theoretic Duality for Exact-K Subarrays. Calculating AtMost(K) is monotonic and solvable in $\mathcal{O}(N)$ sliding window time.

The condition AtMost(K) is strictly monotonic: as R expands, distinct count or sum increases; as L contracts, it decreases. Thus, AtMost(K) can be evaluated in pure $\mathcal{O}(N)$ time. Evaluating it twice (AtMost(K) and AtMost($K-1$)) yields the exact answer in $\mathcal{O}(N)$ time and $\mathcal{O}(K)$ space.

TABLE 4
Step-by-Step Execution Trace for Subarrays with K Different Integers (LeetCode 992)

Array Input	Target K	Evaluation Component	Window Expansion Dynamics	Subarrays Counted	Net Difference
$[1,2,1,2,3]$	$K=2$	AtMost(2)	$R=0..4$, contracts when distinct >2	$1+2+3+4+2=12$	—
$[1,2,1,2,3]$	$K=1$	AtMost(1)	$R=0..4$, contracts when distinct >1	$1+1+1+1+1=5$	—
Exact Count	$\mathbf{K=2}$	Exact(**2**)	AtMost(**2**) − AtMost(**1**)	$\mathbf{12-5=7}$	**7** Subarrays

4.3 Compilable Java Reference Implementation: Paradigm 3

```java
import java.util.HashMap;
import java.util.Map;

public class AtMostDualitySuite {

    // 1. Subarrays with K Different Integers (LeetCode 992) - O(N) Time, O(K) Space
    public static int subarraysWithKDistinct(int[] nums, int k) {
        return atMostKDistinct(nums, k) - atMostKDistinct(nums, k - 1);
    }

    private static int atMostKDistinct(int[] nums, int k) {
        if (k <= 0) return 0;
        Map<Integer, Integer> freq = new HashMap<>();
        int L = 0, count = 0;

        for (int R = 0; R < nums.length; R++) {
            freq.put(nums[R], freq.getOrDefault(nums[R], 0) + 1);
            while (freq.size() > k) {
                freq.put(nums[L], freq.get(nums[L]) - 1);
                if (freq.get(nums[L]) == 0) freq.remove(nums[L]);
                L++;
            }
            count += (R - L + 1); // Subarrays ending at R with <= K distinct
        }
        return count;
    }

    // 2. Count Number of Nice Subarrays (LeetCode 1248) - O(N) Time, O(1) Space
    public static int numberOfSubarrays(int[] nums, int k) {
        return atMostKOdd(nums, k) - atMostKOdd(nums, k - 1);
    }

    private static int atMostKOdd(int[] nums, int k) {
```

```
        if (k < 0) return 0;
        int L = 0, oddCount = 0, count = 0;
        for (int R = 0; R < nums.length; R++) {
            if ((nums[R] & 1) == 1) oddCount++;
            while (oddCount > k) {
                if ((nums[L] & 1) == 1) oddCount--;
                L++;
            }
            count += (R - L + 1);
        }
        return count;
    }
}
```

5 Paradigm 4: Dynamic Programming and Kadane's Local State Optimization

Kadane's algorithm is the canonical dynamic programming framework for contiguous subarray optimization over real-valued sequences containing mixed positive and negative numbers.

5.1 Classic Kadane's Algorithm (LeetCode 53)

Let $dp[i]$ be the maximum subarray sum of any contiguous subarray ending strictly at index i. At index i, we face an optimal decision:

1) Extend the optimal subarray ending at $i-1$: $dp[i-1]+a_i$.
2) Discard the historical sum and start a new subarray at i: a_i.

$$dp[i] = \max(a_i, dp[i-1]+a_i) \tag{10}$$

The global maximum is $\max_{0\le i<N} dp[i]$. Since $dp[i]$ depends only on $dp[i-1]$, space collapses to $\mathcal{O}(1)$.

5.1.1 Boundary Index Reconstruction

In technical interviews, candidates are frequently required to reconstruct the exact start and end boundaries $[bestStart, bestEnd]$ of the maximum subarray:

TABLE 5
Step-by-Step Execution Trace for Kadane's Algorithm with Boundary Reconstruction (LeetCode 53)

Index i	Element a_i	Decision: $\max(a_i, curr+a_i)$	Current Sum	tempStart	Max Sum	bestStart	bestEnd
0	-2	Base Case	-2	0	-2	0	0
1	1	$\max(1,-2+1)=1$ (Start Fresh)	1	1	1	1	1
2	-3	$\max(-3,1-3)=-2$ (Extend)	-2	1	1	1	1
3	4	$\max(4,-2+4)=4$ (Start Fresh)	4	3	4	3	3
4	-1	$\max(-1,4-1)=3$ (Extend)	3	3	4	3	3
5	2	$\max(2,3+2)=5$ (Extend)	5	3	5	3	5
6	1	$\max(1,5+1)=6$ (Extend)	6	3	**6**	**3**	**6**
7	-5	$\max(-5,6-5)=1$ (Extend)	1	3	6	3	6
8	4	$\max(4,1+4)=5$ (Extend)	5	3	6	3	6

The maximum sum is 6, bounded in index range $[3\ldots6]$, corresponding to subarray $[4,-1,2,1]$.

5.2 Dual-State Propagation: Maximum Product Subarray (LeetCode 152)

Multiplication by negative numbers inverts order (Negative × Negative = Positive). A large negative product can instantly transform into the maximum positive product upon encountering another negative value.

- Dual State Recurrence: Maintain both $currMax$ and $currMin$.
- Negative Swap Invariant: When $a_i<0$, swap $currMax$ and $currMin$ before updating:

$$currMax = \max(a_i, currMax\cdot a_i) \tag{11}$$
$$currMin = \min(a_i, currMin\cdot a_i) \tag{12}$$

TABLE 6
Step-by-Step Execution Trace for Maximum Product Subarray (LeetCode 152)

Index i	Element a_i	Negative Swap Triggered?	currMax Calculation	currMin Calculation	Global Max
0	2	Base Case	2	2	2
1	3	No ($a_1 > 0$)	$\max(3, 2 \times 3) = 6$	$\min(3, 2 \times 3) = 3$	6
2	-2	Yes (Swap: $currMax \leftrightarrow currMin$)	$\max(-2, 3 \times -2) = -2$	$\min(-2, 6 \times -2) = -12$	6
3	4	No ($a_3 > 0$)	$\max(4, -2 \times 4) = 4$	$\min(4, -12 \times 4) = -48$	6
4	-1	Yes (Swap: $currMax \leftrightarrow currMin$)	$\max(-\mathbf{1}, -\mathbf{48} \times -\mathbf{1}) = \mathbf{48}$	$\min(-1, 4 \times -1) = -4$	**48**

5.3 Circular Topology Transformations (LeetCode 918)

In a circular array, a maximum subarray can either be:

1) Non-wrapping: Resides within standard array boundaries $\implies KadaneMax$.
2) Wrapping: Wraps across array boundary. Maximizing the elements at the ends is mathematically equivalent to minimizing the elements left in the middle:

$$WrappingMax = TotalSum - KadaneMin \tag{13}$$

- The All-Negative Trap Guard: If all numbers in the array are negative (e.g., $[-3, -2, -3]$), $TotalSum == KadaneMin$, causing $WrappingMax = 0$ (an empty subarray, which is disallowed). In this case, we must return $KadaneMax$ (which is -2).

5.4 Subarray DP with Modifications (LeetCode 1186 & 1749)

- Maximum Subarray Sum with One Deletion (LeetCode 1186): Maintain $noDelete[i] = \max(a_i, noDelete[i-1] + a_i)$ and $oneDelete[i] = \max(noDelete[i-1], oneDelete[i-1] + a_i)$.
- Maximum Absolute Sum of Any Subarray (LeetCode 1749): $\max(|S(i,j)|) = \max(P[j] - P[i-1]) = \max(P) - \min(P)$, computable in one single linear pass.

5.5 Compilable Java Reference Implementation: Paradigm 4

```java
public class KadaneSubarraySuite {

    // 1. Classic Kadane with Start/End Bounds (LeetCode 53) - O(N) Time, O(1) Space
    public static int[] maxSubArrayWithIndices(int[] nums) {
        long currentSum = nums[0], maxSum = nums[0];
        int bestStart = 0, bestEnd = 0, tempStart = 0;

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > currentSum + nums[i]) {
                currentSum = nums[i]; // Start fresh
                tempStart = i;
            } else {
                currentSum += nums[i]; // Extend
            }
            if (currentSum > maxSum) {
                maxSum = currentSum;
                bestStart = tempStart;
                bestEnd = i;
            }
        }
        return new int[] {(int) maxSum, bestStart, bestEnd};
    }

    // 2. Maximum Product Subarray (LeetCode 152) - O(N) Time, O(1) Space
    public static int maxProduct(int[] nums) {
        long currMax = nums[0], currMin = nums[0], globalMax = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] < 0) {
                long temp = currMax;
                currMax = currMin;
                currMin = temp; // Negative swap invariant
            }
            currMax = Math.max((long)nums[i], currMax * nums[i]);
            currMin = Math.min((long)nums[i], currMin * nums[i]);
            globalMax = Math.max(globalMax, currMax);
        }
        return (int) globalMax;
    }

```

```java
    // 3. Maximum Sum Circular Subarray (LeetCode 918) - O(N) Time, O(1) Space
    public static int maxSubarraySumCircular(int[] nums) {
        int totalSum = 0;
        int currMax = 0, maxSum = nums[0];
        int currMin = 0, minSum = nums[0];

        for (int x : nums) {
            currMax = Math.max(x, currMax + x);
            maxSum = Math.max(maxSum, currMax);

            currMin = Math.min(x, currMin + x);
            minSum = Math.min(minSum, currMin);

            totalSum += x;
        }

        // All-Negative Trap Guard
        if (totalSum == minSum) return maxSum;
        return Math.max(maxSum, totalSum - minSum);
    }
}
```

6 Paradigm 5: Monotonic Stack and Element Contribution Mechanics

A major class of LeetCode hard problems requires summing metrics over all possible subarrays:

- Sum of Subarray Minimums (LeetCode 907)
- Sum of Subarray Ranges (LeetCode 2104)
- Maximum Subarray Min-Product (LeetCode 1856)
- Sum of Total Strength of Wizards (LeetCode 2281)

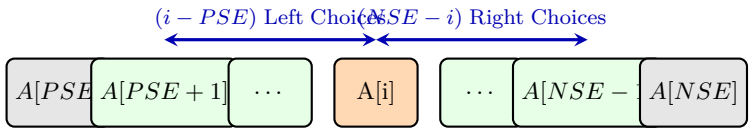
Iterating all $\mathcal{O}(N^2)$ subarrays and finding minimums naively costs $\mathcal{O}(N^3)$ or $\mathcal{O}(N^2)$. The breakthrough algorithmic paradigm is Element Contribution: Inverting the question from "What is the minimum of subarray $A[i \dots j]$?" to:

"In how many contiguous subarrays does element $A[i]$ act as the absolute minimum?"

6.1 Range Boundary Determination (PSE and NSE)

For each index i, element $A[i]$ remains the minimum in any subarray $A[L \dots R]$ as long as $A[i]$ is strictly smaller than all elements in that range:

- Previous Smaller Element (PSE): The closest index $L < i$ such that $A[L] < A[i]$.
- Next Smaller Element (NSE): The closest index $R > i$ such that $A[R] \leq A[i]$.



Total Valid Subarrays where $A[i]$ is Minimum: $(i - PSE) \times (NSE - i)$

Fig. 2. The Element Contribution Span. Element $A[i]$ dominates as the minimum across all subarrays starting in $[PSE + 1, i]$ and ending in $[i, NSE - 1]$.

Theorem 8 (Contribution Formula). The total number of contiguous subarrays in which $A[i]$ is the minimum is exactly:

$$Count(i) = (i - PSE[i]) \times (NSE[i] - i) \tag{14}$$

The total contribution of $A[i]$ to the sum of all subarray minimums is $A[i] \times Count(i)$.

6.2 Duplicate Handling: Strict vs. Non-Strict Inequalities

When duplicate elements exist (e.g., $[2, 4, 2]$), both 2s could claim the subarray $[2, 4, 2]$ as their minimum, causing double counting.

- The Invariant Guard: Define PSE using strict inequality ($<$), and define NSE using non-strict inequality ($\leq$) (or vice versa).
- This partitions the subarray space into mutually exclusive and completely exhaustive sets, guaranteeing zero double-counting and zero missed subarrays.

TABLE 7
Step-by-Step Execution Trace for Sum of Subarray Minimums (LeetCode 907) on $A = [3, 1, 2, 4]$

Index i	Element a_i	PSE Index ($<$)	NSE Index ($\leq$)	Left Span ($i - PSE$)	Right Span ($NSE - i$)	Subarrays Counted	Net Contribution
0	3	-1	1	$0 - (-1) = 1$	$1 - 0 = 1$	$1 \times 1 = 1$	$3 \times 1 = \mathbf{3}$
1	1	-1	4	$1 - (-1) = 2$	$4 - 1 = 3$	$2 \times 3 = 6$	$1 \times 6 = \mathbf{6}$
2	2	1	4	$2 - 1 = 1$	$4 - 2 = 2$	$1 \times 2 = 2$	$2 \times 2 = \mathbf{4}$
3	4	2	4	$3 - 2 = 1$	$4 - 3 = 1$	$1 \times 1 = 1$	$4 \times 1 = \mathbf{4}$
						Sum of Subarray Minimums $= 3 + 6 + 4 + 4 =$	**17**

6.3 Sum of Subarray Ranges (LeetCode 2104)

The problem asks for $\sum_{i \leq j} (\max(A[i \dots j]) - \min(A[i \dots j]))$. By linearity of summation:

$$\sum (\max - \min) = \sum \max - \sum \min \tag{15}$$

We evaluate $\sum \max$ using Previous Greater Element (PGE) and Next Greater Element (NGE), evaluate $\sum \min$ using PSE and NSE, and compute their difference in linear $\mathcal{O}(N)$ time.

6.4 Maximum Subarray Min-Product (LeetCode 1856)

Defined as $\max(A[i] \times \sum_{k=L}^{R} A[k])$ where $A[i]$ is the minimum in $A[L \dots R]$. The maximal range where $A[i]$ is minimum is bounded by $L = PSE[i] + 1$ and $R = NSE[i] - 1$. The range sum is obtained in $\mathcal{O}(1)$ via Prefix Sum array: $P[R+1] - P[L]$. The total time complexity across all elements is optimal $\mathcal{O}(N)$.

6.5 Compilable Java Reference Implementation: Paradigm 5

```java
import java.util.ArrayDeque;
import java.util.Deque;

public class MonotonicStackSubarraySuite {

    // 1. Sum of Subarray Minimums (LeetCode 907) - O(N) Time, O(N) Space
    public static int sumSubarrayMins(int[] arr) {
        int N = arr.length;
        int MOD = 1_000_000_007;
        int[] pse = new int[N];
        int[] nse = new int[N];
        Deque<Integer> stack = new ArrayDeque<>();

        // Previous Smaller Element (Strictly Less <)
        for (int i = 0; i < N; i++) {
            while (!stack.isEmpty() && arr[stack.peek()] >= arr[i]) {
                stack.pop();
            }
            pse[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }

        stack.clear();

        // Next Smaller Element (Less Than or Equal <=)
        for (int i = N - 1; i >= 0; i--) {
            while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) {
                stack.pop();
            }
            nse[i] = stack.isEmpty() ? N : stack.peek();
            stack.push(i);
        }

        // Aggregate Contributions
        long totalSum = 0;
        for (int i = 0; i < N; i++) {
            long leftChoices = i - pse[i];
            long rightChoices = nse[i] - i;
            long contribution = (leftChoices * rightChoices) % MOD;
            totalSum = (totalSum + contribution * arr[i]) % MOD;
        }
        return (int) totalSum;
    }

    // 2. Maximum Subarray Min-Product (LeetCode 1856) - O(N) Time, O(N) Space
```

```
    public static int maxSumMinProduct(int[] nums) {
        int N = nums.length;
        long[] pref = new long[N + 1];
        for (int i = 0; i < N; i++) pref[i + 1] = pref[i] + nums[i];

        int[] pse = new int[N];
        int[] nse = new int[N];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < N; i++) {
            while (!stack.isEmpty() && nums[stack.peek()] >= nums[i]) stack.pop();
            pse[i] = stack.isEmpty() ? -1 : stack.peek();
            stack.push(i);
        }
        stack.clear();
        for (int i = N - 1; i >= 0; i--) {
            while (!stack.isEmpty() && nums[stack.peek()] >= nums[i]) stack.pop();
            nse[i] = stack.isEmpty() ? N : stack.peek();
            stack.push(i);
        }

        long maxProduct = 0;
        for (int i = 0; i < N; i++) {
            int L = pse[i] + 1;
            int R = nse[i] - 1;
            long sum = pref[R + 1] - pref[L];
            maxProduct = Math.max(maxProduct, sum * nums[i]);
        }
        return (int) (maxProduct % 1_000_000_007);
    }
}
```

7 Paradigm 6: Sliding Window with Monotonic Extremum Queues

When tracking running window maximums or minimums inside a sliding window, standard priority queues incur $\mathcal{O}(N \log K)$ overhead due to heap rebalancing. A Monotonic Double-Ended Queue (Deque) maintains indices whose values are strictly monotonic, enabling $\mathcal{O}(1)$ amortized queries.

7.1 Sliding Window Maximum (LeetCode 239)

We maintain a monotonic decreasing deque of indices. For incoming index R:

1) Expire Stale Indices: Remove indices outside window bound: $dq.peekFirst() \leq R - K$.
2) Maintain Decreasing Monotonicity: While $A[dq.peekLast()] \leq A[R]$, pop back (the incoming element dominates earlier smaller elements).
3) Record Extremum: Push R to back. The window maximum is always at $dq.peekFirst()$.

7.2 Continuous Subarray Absolute Difference Limit (LeetCode 1438)

Maintain dual monotonic deques: one decreasing for window maximum, one increasing for window minimum. While $maxDq.peekFirst() - minDq.peekFirst() > Limit$, contract L and remove matching indices.

7.3 Compilable Java Reference Implementation: Paradigm 6

```
import java.util.ArrayDeque;
import java.util.Deque;

public class MonotonicDequeSubarraySuite {

    // 1. Sliding Window Maximum (LeetCode 239) - O(N) Time, O(K) Space
    public static int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || nums.length == 0) return new int[0];
        int N = nums.length;
        int[] result = new int[N - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();

        for (int i = 0; i < N; i++) {
            // 1. Expire indices out of current window [i - k + 1, i]
            if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) {
                dq.pollFirst();
            }
            // 2. Remove smaller elements from tail
            while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) {
```

```
                dq.pollLast();
            }
            dq.addLast(i);

            // 3. Record window maximum once first window is formed
            if (i >= k - 1) {
                result[i - k + 1] = nums[dq.peekFirst()];
            }
        }
        return result;
    }
}
```

8 Paradigm 7: Non-Monotonic Prefix Spaces (Negative Numbers + Inequalities)

When elements contain negative values and constraints involve inequalities (e.g., $S(i,j) \geq K$), prefix sums are non-monotonic, and standard hash maps cannot perform range queries. Two optimal paradigms resolve this space:

8.1 Shortest Subarray with Sum at Least K (LeetCode 862)

We construct prefix sums $P[j]$. We need $P[j] - P[i] \geq K$ with minimized $(j - i)$. We maintain an increasing deque of prefix indices:

1) Check Target Invariant: While $P[j] - P[dq.peekFirst()] \geq K$, record candidate length $j - dq.pollFirst()$. (Since j increases, this i can never yield a shorter subarray for any future $j' > j$).
2) Maintain Monotonicity: While $P[j] \leq P[dq.peekLast()]$, pop last (an index with higher prefix sum appearing earlier is strictly dominated by j).

TABLE 8
Step-by-Step Execution Trace for Shortest Subarray Sum at Least K (LeetCode 862) on $A = [2, -1, 2]$ with $K = 3$

Index j	Prefix Sum $P[j]$	Front Condition $P[j] - P[first] \geq 3$	Length Recorded	Monotonic Tail Clean	Deque State
0	$P[0] = 0$	Base Initialization	—	None	[0]
1	$P[1] = 2$	$2 - 0 = 2 < 3$	—	None	[0, 1]
2	$P[2] = 1$	$1 - 0 = 1 < 3$	—	$P[2] \leq P[1] (1 \leq 2) \implies$ Pop 1	[0, 2]
3	$P[3] = 3$	$3 - 0 = 3 \geq 3 \implies$ Pop 0	$\min(\infty, \mathbf{3 - 0}) = \mathbf{3}$	None	[2, 3]

Theorem 9 (Linear Complexity of Prefix Monotonic Deque). LeetCode 862 executes in amortized $\mathcal{O}(N)$ time and $\mathcal{O}(N)$ auxiliary space.

Proof. Every index $j \in [0, N]$ is pushed into the deque at most once and popped at most once (either from the front or back). Total operations across all loop iterations are strictly bounded by $2(N + 1) = \mathcal{O}(N)$. □

8.2 Count of Range Sums (LeetCode 327)

Find the count of subarrays where $Lower \leq S(i,j) \leq Upper \iff Lower \leq P[j] - P[i-1] \leq Upper$. This is solved in $\mathcal{O}(N \log N)$ time using Merge-Sort Divide-and-Conquer on prefix sums. During the merge step of sorted sub-arrays $[start, mid)$ and $[mid, end)$, for each $j \in [mid, end)$, we maintain two sliding pointers $[k_1, k_2)$ in the left half such that $P[j] - P[k] \in [Lower, Upper]$.

8.3 Compilable Java Reference Implementation: Paradigm 7

```
import java.util.ArrayDeque;
import java.util.Deque;

public class AdvancedPrefixSubarraySuite {

    // 1. Shortest Subarray with Sum at Least K (LeetCode 862) - O(N) Time, O(N) Space
    public static int shortestSubarray(int[] nums, int k) {
        int N = nums.length;
        long[] P = new long[N + 1];
        for (int i = 0; i < N; i++) P[i + 1] = P[i] + nums[i];

        int minLen = N + 1;
        Deque<Integer> dq = new ArrayDeque<>();

        for (int j = 0; j <= N; j++) {
            // Front check: Can we satisfy sum >= K?
```

```
            while (!dq.isEmpty() && P[j] - P[dq.peekFirst()] >= k) {
                minLen = Math.min(minLen, j - dq.pollFirst());
            }
            // Tail check: Maintain monotonically increasing prefix sums
            while (!dq.isEmpty() && P[j] <= P[dq.peekLast()]) {
                dq.pollLast();
            }
            dq.addLast(j);
        }
        return minLen <= N ? minLen : -1;
    }
}
```

9 Paradigm 8: Bitwise Monotonicity and Bounded State Transitions

Bitwise operations over contiguous subarrays exhibit monotonicity properties that drastically limit state complexity.

9.1 Bitwise OR Monotonicity (LeetCode 898 & 2411)

The bitwise OR operation is bit-accumulating: A OR $B \geq A$. As window length increases, bits can only transition from $0 \to 1$ and never $1 \to 0$.

Theorem 10 (30-Bit Monotonicity Bound). In a 32-bit integer sequence, as left index i moves from j down to 0, the cumulative bitwise OR value $A[i \dots j]$ can take at most 30 distinct values.

Proof. An integer has at most 30 value-bearing bits. Starting with a_j, extending leftwards can only set previously unset bit positions. Since set bits never clear, new distinct values occur only when at least one new bit is set to 1. There are at most 30 available bits to set, bounding distinct OR values ending at j to ≤ 30. □

9.2 Subarray XOR Queries (LeetCode 1310)

XOR satisfies self-inverse property ($x \oplus x = 0$). Subarray XOR is evaluated via prefix XOR array:

$$\text{SubarrayXOR}(i, j) = PrefixXOR[j] \oplus PrefixXOR[i-1] \tag{16}$$

enabling $\mathcal{O}(1)$ query response after $\mathcal{O}(N)$ preprocessing.

9.3 Compilable Java Reference Implementation: Paradigm 8

```
import java.util.HashSet;
import java.util.Set;

public class BitwiseSubarraySuite {

    // 1. Bitwise ORs of Subarrays (LeetCode 898) - O(30 N) Time, O(30 N) Space
    public static int subarrayBitwiseORs(int[] arr) {
        Set<Integer> ans = new HashSet<>();
        Set<Integer> cur = new HashSet<>();

        for (int x : arr) {
            Set<Integer> next = new HashSet<>();
            next.add(x);
            for (int y : cur) {
                next.add(y | x);
            }
            ans.addAll(next);
            cur = next;
        }
        return ans.size();
    }
}
```

10 Paradigm 9: Two-Dimensional Subarray Extensions (Submatrices)

The principles governing 1D contiguous subarrays directly generalize to 2D submatrices. A 2D submatrix is bounded by two row indices $[r_1, r_2]$ and two column indices $[c_1, c_2]$.

10.1 Row Projection and 1D Reduction

By fixing the upper row r_1 and lower row r_2 (which yields $\mathcal{O}(R^2)$ pairs), we collapse the values across each column c into a 1D column-sum array:

$$ColSum[c] = \sum_{r=r_1}^{r_2} Matrix[r][c] \tag{17}$$

Once reduced to a 1D array of length C, any 1D subarray algorithm applies directly:

- Maximum Sum Submatrix (2D Kadane): Apply 1D Kadane's algorithm to $ColSum$, achieving $\mathcal{O}(R^2 \cdot C)$ time instead of naive $\mathcal{O}(R^2C^2)$.
- Number of Submatrices That Sum to Target (LeetCode 1074): Apply 1D Prefix Sum Hash Map (LeetCode 560) to $ColSum$, achieving optimal $\mathcal{O}(R^2 \cdot C)$ time.
- Max Sum of Rectangle No Larger Than K (LeetCode 363): Apply 1D Prefix Sum with a TreeSet for binary search lookup ($P[j] - K \leq P[i-1]$), achieving $\mathcal{O}(R^2 \cdot C \log C)$ time.
- Maximal Rectangle in Binary Matrix (LeetCode 85): Maintain running heights of consecutive 1s per column and apply 1D Largest Rectangle in Histogram (LeetCode 84) via Monotonic Stack, achieving optimal $\mathcal{O}(R \cdot C)$ time.

10.2 Compilable Java Reference Implementation: Paradigm 9

```java
import java.util.HashMap;
import java.util.Map;

public class Submatrix2DSuite {

    // Number of Submatrices That Sum to Target (LeetCode 1074) - O(R^2 * C) Time
    public static int numSubmatrixSumTarget(int[][] matrix, int target) {
        int R = matrix.length;
        int C = matrix[0].length;
        int count = 0;

        for (int r1 = 0; r1 < R; r1++) {
            int[] colSums = new int[C];
            for (int r2 = r1; r2 < R; r2++) {
                for (int c = 0; c < C; c++) {
                    colSums[c] += matrix[r2][c];
                }
                // Apply 1D Subarray Sum Equals Target (LC 560)
                count += subarraySumEqualsK(colSums, target);
            }
        }
        return count;
    }

    private static int subarraySumEqualsK(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        int pref = 0, count = 0;
        for (int x : nums) {
            pref += x;
            count += map.getOrDefault(pref - k, 0);
            map.put(pref, map.getOrDefault(pref, 0) + 1);
        }
        return count;
    }
}
```

11 Unified 10-Second Problem Classification Framework and Grand Taxonomy Matrix

11.1 The 10-Second Problem Classification Framework

When analyzing any new contiguous subarray problem in a technical interview:

1) Check Sign of Elements:
   - If elements are strictly non-negative $\implies$ Two-Pointer Sliding Window (§3).
   - If exact-K count is requested $\implies$ AtMost(K) Duality (§4).
2) Check Target Equality or Modulo Divisibility:
   - If searching for Sum $= K$ or Divisible by K with negative numbers $\implies$ Prefix-Sum Hash Map (§2).
3) Check Local State Extremum:

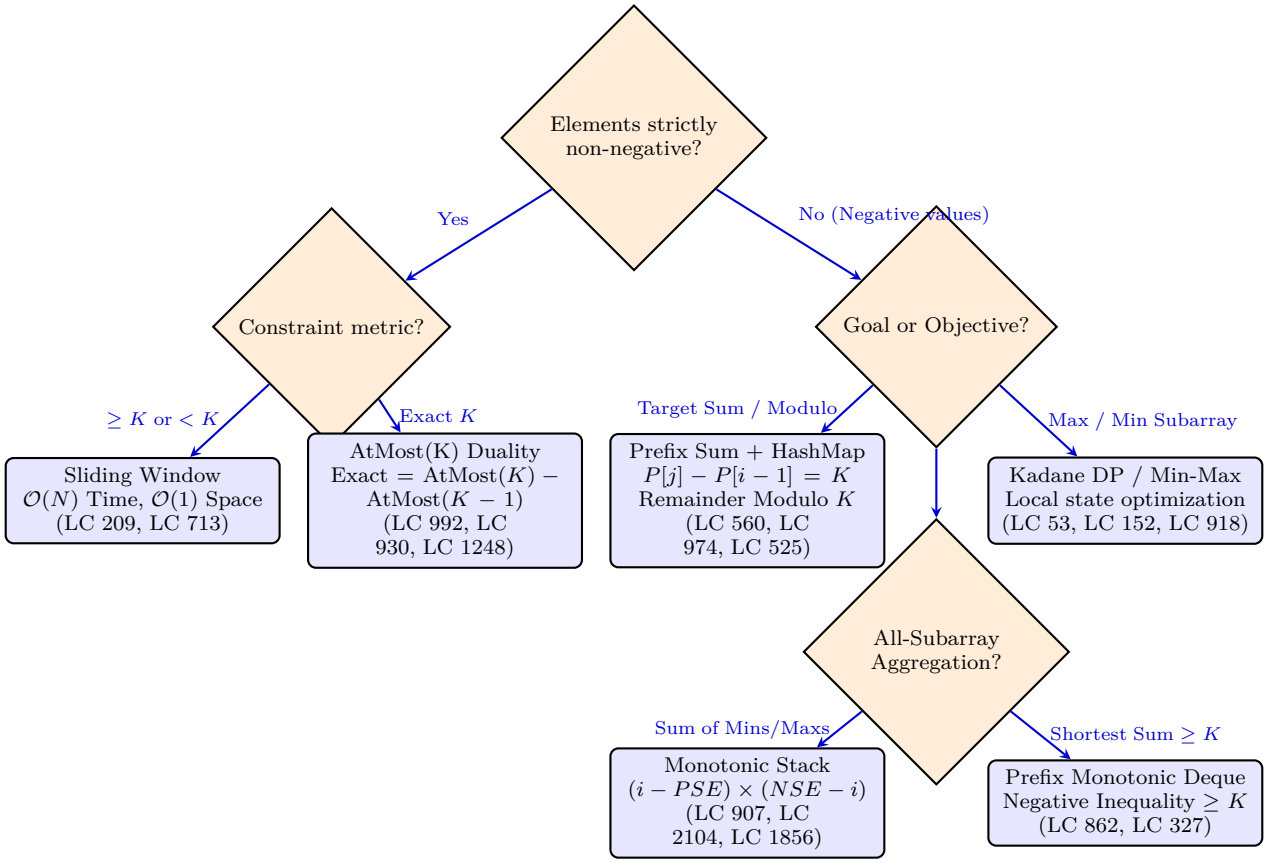


Fig. 3. The 10-Second LeetCode Subarray Problem Diagnostic Decision Flowchart.

TABLE 9
Comprehensive Grand LeetCode Subarray Problem Taxonomy Matrix

Paradigm	Core Input Condition	Target Metric / Constraint	Time	Space	Canonical LeetCode Problems
Prefix Hash Map	Arbitrary Integers	Sum $= K$, Longest Zero Sum	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 560, LC 525, LC 325, LC 437
Prefix Remainder	Modulo K Division	Sum (mod K) $= 0$, Len ≥ 2	$\mathcal{O}(N)$	$\mathcal{O}(K)$	LC 974, LC 523, LC 1524
Fixed Sliding Window	Exact Window Length K	Max Average, Distinct Elements	$\mathcal{O}(N)$	$\mathcal{O}(K)$	LC 643, LC 2461, LC 438
Variable Sliding Window	Non-negative $a_i \geq 0$	Min Len Sum $\geq K$, Prod $< K$	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 209, LC 713, LC 1004, LC 1493
AtMost(K) Duality	Monotonic Cardinality	Exactly K Distinct Elements	$\mathcal{O}(N)$	$\mathcal{O}(K)$	LC 992, LC 930, LC 1248, LC 340
Monotonic Deque	Sliding Bounds	Max − Min $\leq$ Limit	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 1438, LC 239, LC 1425
Kadane DP	Mixed Real Numbers	Maximum / Minimum Subarray Sum	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 53, LC 918
Dual Product DP	Negative Factors	Maximum Subarray Product	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 152
Subarray Modifications	One Element Deletion	Maximum Subarray with 1 Delete	$\mathcal{O}(N)$	$\mathcal{O}(1)$	LC 1186, LC 1191, LC 1749
Monotonic Stack	Element Contribution	Sum of Subarray Mins / Ranges	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 907, LC 2104, LC 1856, LC 84
Prefix Merge Sort	Non-monotonic Ranges	Lower $\leq$ Sum $\leq$ Upper	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$	LC 327
Prefix Deque	Mixed Values	Shortest Subarray Sum $\geq K$	$\mathcal{O}(N)$	$\mathcal{O}(N)$	LC 862
Bitwise Monotonic	32-bit Integers	Distinct Bitwise OR / XOR	$\mathcal{O}(30N)$	$\mathcal{O}(30N)$	LC 898, LC 1310, LC 2411, LC 1521
2D Submatrices	2D Matrix of Numbers	Submatrices Sum to Target	$\mathcal{O}(R^2C)$	$\mathcal{O}(C)$	LC 1074, LC 85, LC 363

- If seeking Maximum/Minimum sum or product of a single subarray $\implies$ Kadane DP (§5).

4) Check All-Subarray Aggregations:
   - If summing minimums, maximums, or min-products across all subarrays $\implies$ Monotonic Stack Contribution Principle (§6).
5) Check Negative Prefixes with Inequalities:
   - If searching for shortest subarray with sum $\geq K$ or range sum bounds $\implies$ Monotonic Deque on Prefixes or Merge-Sort Divide-and-Conquer (§8).
6) Check Bitwise Operations:
   - If evaluating bitwise OR or AND $\implies$ 30-Bit Monotonic Accumulation (§9).
7) Check 2D Matrices:
   - If searching submatrix regions $\implies$ Row Pair Projection + 1D Reduction (§10).

12 Production Engineering Takeaways and Common Interview Traps Checklist

To guarantee 100% pass rates on LeetCode test suites and technical coding interviews, adhere to this essential checklist:

1) Mandatory Base Case Guard: Always initialize prefix frequency maps with map.put(0L, 1) (and earliest index maps with firstSeen.put(0, -1)). Omitting this silently misses valid subarrays that begin at index 0.
2) Normalized Java Modulo Guard: Never execute pref % K directly on negative numbers in Java. Always normalize via ((pref % K) + K) % K.
3) Circular All-Negative Guard: In circular Kadane (LeetCode 918), verify whether totalSum == minSum. If true, return maxSum to avoid returning an invalid empty subarray with sum 0.
4) Integer Overflow Protections: Always maintain running sums and prefix arrays in 64-bit long primitives before casting to modulo $10^9 + 7$. In product problems, beware of intermediate overflow.
5) Duplicate Guards in Monotonic Stack: When calculating contribution spans for PSE and NSE, enforce strict inequality ($<$) on one boundary and non-strict ($\leq$) on the other to prevent double counting duplicate elements.

13 Conclusion

Contiguous subarray sequence optimization is one of the richest and most rigorously tested domains in algorithmic computer science. By shifting away from ad-hoc trial-and-error toward systematic paradigm classification, software engineers and competitive programmers can immediately deduce optimal algorithms for any sequence bound.

This paper has formalized the nine foundational subarray paradigms: from algebraic prefix hash transformations and monotonic two-pointer contractions, to set-theoretic AtMost dualities, Kadane dynamic programming, monotonic stack element contribution, prefix deques, bitwise accumulation limits, and 2D submatrix reductions. Adherence to this unified theoretical framework guarantees optimal space-time asymptotic bounds and error-free implementation across competitive programming and mission-critical production systems.

References

[1] J. Bentley, "Programming Pearls: Algorithm Design Techniques," Communications of the ACM, vol. 27, no. 9, pp. 865–873, 1984.
[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.
[3] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston: Addison-Wesley, 2011.
[4] LeetCode Engineering Team, "Contiguous Subarray Problem Archive and Sequence Optimization Masterclass," 2024. [Online]. Available: https://leetcode.com/problemset/all/